PROJECT DOCUMENTATION

Personal Finance Bot

Telegram + Google Sheets + Gemini AI + Web Dashboard

Main repository	enefisualcode/bot-keuangan-v2
Web dashboard	enefisualcode/keuangan-web

Documented version: bot commit 90202a9 (Sep 2, 2026) | web commit c7db0ae (Aug 30, 2026)

This document describes the system design, workflow, features, setup, data structure, security, and development opportunities. Personal data from the source workbook is not reproduced.

1. Project Overview

Personal Finance Bot is a multi-user financial tracking system operated through Telegram. Each user can connect their own Google Spreadsheet, record expenses and income, upload receipt photos/PDFs for AI extraction, request summaries, and ask AI questions about transaction patterns. The data is then visualized through a Google Sheets dashboard and a lightweight web dashboard.

The main goal of this project is to make financial tracking feel like a normal conversation, without requiring users to open a spreadsheet or accounting app every time they want to record a transaction.

Core value

- Fast input through Telegram: commands, natural language, receipt photos, or PDFs.
- Multi-user: each user has their own spreadsheet; the mapping is stored in a master sheet.
- AI-assisted: Gemini reads receipts, interprets purchase text, and analyzes transaction history.
- The data remains easy to audit because Google Sheets is the primary data source.
- A mobile-friendly web dashboard shows period summaries, trends, Pay Later, savings, and investments.

2. System Components

Component	Technology	Role
Telegram Bot	Python, python-telegram-bot 22.8	Primary interface for transaction input, summaries, AI, and user administration.
Data Store	Google Sheets + gspread	Stores expenses, income, dashboard data, charts, and user mappings.
AI Engine	Gemini 2.5 Flash	Receipt OCR/interpretation, natural-language parsing, and financial Q&A.
Sheets Authentication	Google Service Account	Gives the bot access to

		spreadsheets shared by users.
Web Dashboard	HTML/CSS/Vanilla JS	Mobile-first visualization, dark/light themes, periods, charts, and data refresh.
Web data endpoint	Google Apps Script / JSONP	Bridges Google Sheets data to the web dashboard.
Bot runtime	Worker process: python bot.py	Suitable for VPS/Railway or another host that runs a persistent Python process.

3. Architecture & Data Flow

```
User -> Telegram -> bot.py -> Google Sheets (user)
                    |-> Gemini AI
                    |-> Master Sheet / Users
Google Sheets -> Apps Script JSONP -> keuangan-web/index.html -> Browser Dashboard
```

User onboarding flow

1. The user opens the bot and runs /start.
2. If no spreadsheet is connected yet, the user can use /daftar <spreadsheet link> or /buatkan <email>.
3. The bot ensures that the service account has Editor access.
4. The bot creates/prepares the Pengeluaran, Pemasukan, Dashboard, and Grafik sheets.
5. The Telegram user ID -> spreadsheet ID mapping is stored in the Users sheet of the master spreadsheet.
6. After that, all user transactions are routed to the user's own spreadsheet.

Receipt logging flow

7. The user sends a receipt photo or PDF document.
8. Gemini extracts the date, merchant, category, amount, notes, and purchased items.
9. The bot validates the result, including detecting multiple transactions in one file and transactions with dates that are too old.
10. The user reviews the extracted result and confirms it using an inline button.
11. After confirmation, the bot asks for/determines the payment type and writes the transaction to Google Sheets.

4. Main Bot Features

Feature / Command	Function
/start	Usage guide and main keyboard.
/daftar <link/id>	Connects the user's spreadsheet.
/buatkan <email>	Creates a new spreadsheet and shares it with the user's email.
/catat	Records an expense manually.
/masuk	Records income manually.
Natural language	Example: "jajan bakso 20rb di warung Ali"; AI understands the transaction without a command.
Receipt photo/PDF	Automatically reads transaction evidence using Gemini.
/rekap	Daily, specific-date, monthly/period summaries,

	including payment-type filters.
/tanya	AI analyzes the user's transaction history.
/hapus	Deletes a transaction through the bot workflow.
/info	Shows the connected spreadsheet/dashboard.
/perbaruidashboard	Refreshes the Google Sheets dashboard structure.
/fitur	Shows the feature list.
/saran	Sends user feedback to the developer.
/myid	Displays the user's Telegram ID.
/umumkan	Developer/admin-only broadcast.
/statususer	Checks user activity level; developer/admin only.

Standard expense categories

The bot normalizes transactions into the system categories: Makan (Food), Donasi (Donation), Transport, Belanja (Shopping), Hiburan (Entertainment), Tagihan (Bills), Investasi (Investment), Tabungan (Savings), and Lainnya (Other). Normalization uses keywords from the category, merchant, and transaction context.

5. Google Sheets Data Structure

The reviewed source workbook contains five sheets: Dashboard, Pengeluaran, Users, Pemasukan, and Grafik. This documentation shows only the schema, not personal data.

Pengeluaran Sheet (Expenses)

Column	Content
Tanggal (Date)	Transaction date.
Kategori (Category)	Category from input/normalization.
Nominal (Amount)	Expense amount in Rupiah.
Merchant	Merchant name, when available.
Sumber (Source)	Example: Manual or Struk (Receipt).
Catatan (Notes)	User notes/caption.
Tipe Bayar (Payment Type)	Payment method or type.
Barang (Items)	List of items extracted from the receipt, when available.

Pemasukan Sheet (Income)

Column	Content
Tanggal (Date)	Income date.
Kategori (Category)	Income category/type.
Nominal (Amount)	Income amount.
Sumber (Source)	Input source.
Catatan (Notes)	Additional notes.

Users Sheet (master)

Column	Content
user_id	Telegram user ID.
username	Telegram username, when available.
spreadsheet_id	ID of the user's spreadsheet.
tanggal_daftar	Registration timestamp.

Security note: the Users sheet contains sensitive operational identifiers. Do not publish it in the repository or portfolio screenshots.

6. Web Dashboard

The keuangan-web repository contains a single-page dashboard (index.html) without a framework. The page retrieves data through Google Apps Script using JSONP, allowing it to run both on the web and locally without being blocked by file:// fetch restrictions.

- Summary of today's expenses and comparison with the average.
- Total Pay Later amount as a reminder of deferred spending.
- Income and expense summaries by period.
- Daily expense chart with day selection for transaction details.
- Period selection using a financial cycle from the 25th to the 24th.
- Dedicated Savings & Investment tab with running balance trends.
- Dark / light / system theme.
- Pull-to-refresh on touch devices and automatic polling every 45 seconds while the page is visible.
- Refresh when the browser tab becomes active again.

Important dashboard configuration

```
const URL_DATA = "<Google Apps Script Web App URL>";
const HARI_SIKLUS = 25;
const SALDO_AWAL_TABUNGAN = <initial balance>;
```

The SALDO_AWAL_TABUNGAN value in the bot and web dashboard must remain consistent so that cumulative savings calculations do not diverge.

7. Environment Variables & Setup

Bot environment variables

Variable	Required?	Description
TELEGRAM_BOT_TOKEN	Yes	Bot token from BotFather.
GEMINI_API_KEY	Yes	API key for Gemini.
MASTER_SPREADSHEET_ID	Yes	Master spreadsheet containing the user list.
GOOGLE_CREDENTIALS_JSON	Yes under current validation	Service account credential JSON.
GOOGLE_CREDENTIALS_FILE	Runtime alternative	Credential file path; the code supports it, but the current cek_konfigurasi still requires GOOGLE_CREDENTIALS_JSON.
ADMIN_ID	Recommended	Developer Telegram ID for broadcasts, feedback, and user status.
USERS_SHEET	Optional	Default: Users.
SHEET_PENGELUARAN	Optional	Default: Pengeluaran.
SHEET_PEMASUKAN	Optional	Default: Pemasukan.
SHEET_DASHBOARD	Optional	Default: Dashboard.
SHEET_GRAFIK	Optional	Default: Grafik.

Dependencies

```
python-telegram-bot==22.8
gspread==6.2.1
```

```
oauth2client==4.1.3
google-generativeai==0.8.5
```

Bot deployment steps

12. Create a Telegram bot and obtain the token from BotFather.
13. Create a Google Cloud service account and JSON credentials for Sheets/Drive access.
14. Create the master spreadsheet, then make sure the service account has access.
15. Configure the environment variables on the VPS/Railway.
16. Install dependencies from requirements.txt.
17. Run the worker: python bot.py. The repository Procfile also defines "worker: python bot.py".
18. Test /start, spreadsheet onboarding, manual logging, receipt reading, /rekap, and /tanya.

8. Reliability & Error Handling

- The Google Sheets connection reuses a single gspread client so authentication is not repeated for every request.
- Certain operations use retry with exponential backoff for network failures, HTTP 429 responses, and 5xx server errors.
- Dates use WIB (UTC+7), keeping results consistent even if the server runs in UTC.
- The bot validates unclear receipt files, multiple receipts in one file, and transaction dates that are too old.
- The web dashboard has a 20-second timeout and a fallback message if the Apps Script endpoint cannot be reached.

9. Security & Privacy

- Do not commit TELEGRAM_BOT_TOKEN, GEMINI_API_KEY, service account credentials, or the master spreadsheet to GitHub.
- User spreadsheets should be shared only with the service account actually used by the system.
- Assume the dashboard endpoint may expose data if its URL is public; consider authentication or tokens if the dashboard is used by multiple users.
- The AI service receives transaction data for OCR and analysis. Users should understand that analyzed transactions are sent to an AI model service.
- User data is separated by spreadsheet_id, but service account access remains the primary trust boundary.
- The Users sheet must not be published because it contains Telegram IDs and spreadsheet IDs.

10. Technical Issues to Improve

Finding	Impact	Recommendation
cek_konfigurasi() requires GOOGLE_CREDENTIALS_JSON even though the code also supports GOOGLE_CREDENTIALS_FILE.	A deployment using only a credential file can fail during startup.	Change validation to accept either JSON or FILE.
SALDO_AWAL_TABUNGAN is defined in both bot.py and index.html.	The values may become inconsistent when only one is changed.	Move it to a single configuration source/API.
The dashboard uses a hard-coded Apps Script URL in index.html.	Less flexible for multi-user setups or different environments.	Use build/runtime configuration or per-user parameters.

Both repositories have very minimal README files.	Harder for potential collaborators/clients to understand.	Add a quick start, architecture, environment variables, screenshots, and a demo.
oauth2client is a legacy library.	Not ideal for long-term maintenance.	Consider migrating to google-auth / gspread service_account.
The dashboard and bot are in separate repositories.	Versions can drift and configuration is duplicated.	Add a release/version convention or move to a monorepo if the project grows.

11. Repository Structure

bot-keuangan-v2

```
bot-keuangan-v2/
├── bot.py            # Telegram, Sheets, AI, and dashboard setup logic
├── requirements.txt  # Python dependencies
├── Procfile          # worker: python bot.py
├── .gitignore
└── README.md
```

keuangan-web

```
keuangan-web/
├── index.html        # dashboard UI + CSS + JavaScript
├── apple-touch-icon.png
└── README.md
```

12. Minimum Test Scenarios

Area	Test
Onboarding	New user with /daftar using a valid spreadsheet, invalid link, spreadsheet not yet shared, and /buatkan with email.
Manual expense	/catat amount and category; yesterday's date; notes containing delimiters.
Natural language	Enter an informal sentence with an amount such as "20rb" and a merchant.
Receipt OCR	Clear photo, blurry photo, PDF, old receipt, and multiple receipts in one file.
Income	Use /masuk and verify the value is written to Pemasukan.
Summary	Today, a specific date, month/period, and Pay Later filter.
AI Q&A	Questions about frequency, totals, and saving suggestions using transaction history.
Dashboard	Latest data, period switching, savings/investments, theme, refresh, and endpoint failure.
Multi-user	Ensure user A's transactions never enter user B's spreadsheet.
Admin	/umumkan and /statususer can only be run by ADMIN_ID.

13. Development Roadmap

- Split bot.py into modules: config, sheets, ai, handlers, models, and services.
- Add automated tests for amount parsing, categories, dates, and callback flows.
- Add dashboard authentication and a unique dashboard URL for each user.
- Add monthly report export to PDF/Excel.
- Add category budgets and alerts when limits are approaching.
- Add recurring transactions for bills, installments, and automatic savings.
- Add backup/recovery and an audit log for transaction changes.
- Add GitHub Actions CI for linting/testing before deployment.

14. Portfolio Summary

This project demonstrates the ability to build an end-to-end workflow without a native mobile application: Telegram as the input layer, AI for understanding text and documents, Google Sheets as a transparent data store, and a web dashboard for visualization. Its main technical value is integrating multiple services into a simple financial tracking experience for users.